

pulp: Pulp classes

```
lpProblem
lpVariable
lpAffineExpression
lpConstraint
lpConstraint.makeElasticSubProblem
FixedElasticSubProblem
```

Todo: LpFractionConstraint, FractionElasticSubProblem

The LpProblem Class

```

class IpOptProblem(name='IpOptProblem', solver=IpOpt):
    """
    An LP Problem

    Creates an LP Problem

    This function creates a new LP Problem with the specified associated parameters

    Parameters:
    * name -- name of the problem used in the output. Ip file
    * sense -- of the LP problem objective. Either lpminimize (default) or lpmaximize.

    Returns:
    An LP Problem

    Three important attributes of the problem are:

    objective
    The objective of the problem, an lpAffineExpression

    constraints
    An ordered dictionary of constraints of the problem - indexed by their names.

    status
    The return status of the problem from the solver.

    Some of the more important methods:

    solve(solver=None, **kwargs)
    Solve the given LP problem.

    This function changes the problem to make it suitable for solving then calls the solver. actualSolve method to find the solution

    Parameter: solver - Optional: the specific solver to be used, defaults to the default solver.

    Side Effects:
    * The attributes of the problem object are changed in actualSolve() to reflect the LP solution

    roundSolution(eps=1e-11, roundOff=0.0000000000000001e-05, eps=9.9999999999999995e-08)
    Rounds the LP variables

    Inputs:
    * none

    Side Effects:
    * The LP variables are rounded

    setObjective(obj)
    Sets the input variable as the objective function. Used in Columnwise Modeling

    Parameter: obj -- the objective function of type lpConstraint

    Side Effects:
    * The objective function is set

    writeLP(filename, writeSOS=True, mip=False)
    Write the given LP problem to a .lp file.

    This function writes the specifications (objective function, constraints, variables) of the defined LP problem to a file.

    Parameter: filename -- the name of the file to be created.

    Side Effects:
    * The file is created.
    """

```

Variables and Expressions

```
class var name, LLelement(name)
    Base class for VarVariable and LCConstraintVar

class var name, lowerBound=None, upperBound=None, cat=Continuous, n=None
    This class models a VarVariable with the specified associated n
    parameters.

Parameters:
- name – The name of the variable used in the output. Its life
  - lowerBound – The lower bound on this variable's range. Default is negative
  - upperBound – The upper bound on this variable's range. Default is positive
  - cat – The category this variable is in. Integer: Binary or Continuous/default
  - n – Used for column based modeling: relates to variable's existence

addVariableToConstraints(c)
  adds a variable to the constraints indicated by the LCConstraints in n

classListDict names, index, lowerBound=None, upperBound=None, cat=0, indexStart=0
  Creates a dictionary of Var variables

This function creates a dictionary of Var Variables with the specified
associated parameters.

Parameters:
- name – The name of the names of each Var variable created
- index – A list of strings of the keys to the dictionary of Var variables.
  - lowerBound – The lower bound on these variables' range. Default is n
  - upperBound – The upper bound on these variables' range. Default is n
  - cat – The category these variables are in. Integer or Continuous/default
- indexStart – Returns: A dictionary of Var Variables

setInitialValue(val)
  sets the initial value of the Variable to val may or may not be supported by the solver
```

```
Example:
>>> x = lpVariable('x', lowBound = 0, cat='Continuous')
>>> y = lpVariable('y', upBound = 5, cat='Integer')
```

gives $x \in [0, \infty)$, $y \in (-\infty, 5]$ an integer.

[illegible]

sulp.lpSum(vector)
Calculate the sum of a list of linear expressions

Parameter: vector – A list of linear expressions

Constraints

```

class pulp.LpConstraint(*args, sense=None, name=None, rhs=None)
Bases: pulp.pulp.LpAffixExpression

An LP constraint

Parameters:
    e -- an instance of LpAffixExpression
    sense -- one of LpConstraintLT, LpConstraintEQ, LpConstraintGE, LpConstraintLE (0, 1, -1 respectively)
    name -- identifying string
    rhs -- numerical value of constraint target

makeElasticSubProblem(*args, **kwargs)
    Builds an elastic subproblem by adding variables to a hard constraint
    uses FixedElasticSubProblem

```

Elastic Constraints

A constraint $C(x) = c$ (equality may be replaced by $\leq$ or $\geq$) can be elasticized to the form

where D denotes some interval containing the value c .

Define the constraint in two steps

1. instantiate constraint (subclass of `LpConstraint`) with target c .
2. call its `makeElasticSubProblem()` method which returns an object of type `FixedElasticSubProblem` (subclass of `LpProblem`) - its objective is the minimization of the distance of $C(x)$ from D .

```
constraint = LpConstraint(..., rhs = c)
elasticProblem = constraint.makeElasticSubProblem(
    penalty = <penalty_value>,
    proportionFreebound = <freebound_value>,
    proportionFreeboundList = <freebound_list_value>,
```

where:

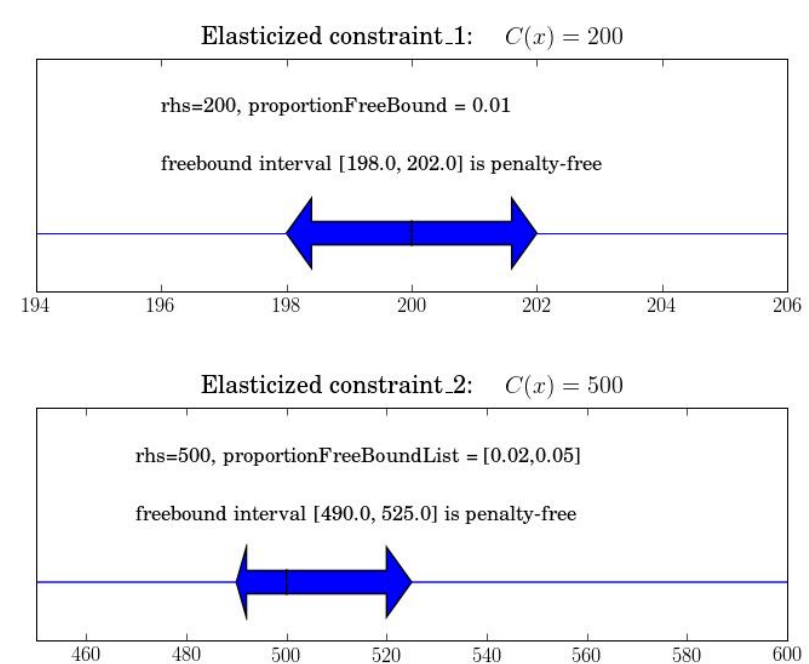
- `<penalty_value>` is a real number
- `<freelbound_value>` $a \in [0, 1]$ specifies a symmetric target interval $D = (c(1 - a), c(1 + a))$ about c
- `<freelbound_list_value>` = $\{a, b\}$, a list of proportions $a, b \in [0, 1]$ specifying an asymmetric target interval $D = (c(1 - a), c(1 + b))$ about c

The penalty applies to the constraint at points x where $C(x) \notin D$. The magnitude of `<penalty_value>` can be assessed by examining the final objective function in the `.ip` file written by `IpProblem.writeIP()`.

Example:

```
>>> constraint_1 = lpConstraint('ex_1',sense=1,rhs=280)
>>> elasticProblem_1 = constraint_1.makeElasticSubproblem(penalty=1, proportionFreeBound = 0.01)
>>> constraint_2 = lpConstraint('ex_2',sense=0,rhs=580)
>>> elasticProblem_2 = constraint_2.makeElasticSubproblem(penalty=1,
proportionFreeBound = 0.01, 0.01)
```

1. constraint_1 has a penalty-free target interval of 1% either side of the rhs value, 200
2. constraint_2 has a penalty-free target interval of - 2% on left and 5% on the right side of the rhs value, 500



Following are the methods of the return-value:

```
class pulp.FixedElasticSubProblem(constraint, penalty=None, proportionFreeBound=None, proportionFreeBoundList=None)
    Bases: pulp.pulp.LpProblem
```

Parameters: *constraint* – The `LpConstraint` that the elastic constraint is based on

- penalty* – penalty applied for violation (+ve or -ve) of the constraints
- proportionFreeBound* – the proportional bound (+ve and -ve) on constraint violation that is free from penalty
- proportionFreeBoundList* – the proportional bound on constraint violation that is free from penalty, expressed as a list where [-ve, +ve]

alterName(name)
 Alters the name of anonymous parts of the problem

`findDifferenceFromRHS()`
The amount the actual value varies from the RHS (sense: LHS - RHS)

findLHSValue()
finds the LHS value of the constraint (without the free variable and/or penalty variable) - assumes the constant is on the rhs

isViolated()
returns true if the penalty variables are non-zero

Combinations and Permutations

`gulp.combination(orgset, k=None)`
returns an iterator that lists the combinations of orgset of length k

Parameters: • *orgset* – the list to be iterated
• *k* – the cardinality of the subsets

Returns: an iterator of the subsets

```
example:
>>> c = combination([1,2,3,4],2)
>>> for s in c:
...     print s
(1, 2)
(1, 3)
(1, 4)
(2, 3)
(2, 4)
```

`sulp.allcombinations(orgset, k)`
returns all permutations of orgset with up to k items

Parameters: • *orgset* – the list to be iterated
• *k* – the maxcardinality of the subsets

Returns: an iterator of the subsets

```
example:
```

```
>>> c = allcombinations([1,2,3,4],2)
>>> for s in c:
...     print s
```

```
(1, )
(2, )
(3, )
(4, )
(1, 2)
(1, 3)
(1, 4)
(2, 3)
```



```
(2, 4)
(3, 4)
```

`pulp.permutation(orgset, k=None)`

returns an iterator that lists the permutations of orgset of length k

Parameters: • `orgset` – the list to be iterated
• `k` – the cardinality of the subsets

Returns: an iterator of the subsets

example:

```
>>> c = permutation([1,2,3,4],2)
>>> for s in c:
...     print s
(1, 2)
(1, 3)
(1, 4)
(2, 3)
(2, 4)
(3, 4)
```

`pulp.allpermutations(orgset, k)`

returns all permutations of orgset with up to k items

Parameters: • `orgset` – the list to be iterated
• `k` – the maxcardinality of the subsets

Returns: an iterator of the subsets

example:

```
>>> c = allpermutations([1,2,3,4],2)
>>> for s in c:
...     print s
(1,)
(2,)
(3,)
(4,)
(1, 2)
(1, 3)
(1, 4)
(2, 3)
(2, 4)
(3, 4)
(1, 2, 3)
(1, 2, 4)
(1, 3, 4)
(2, 3, 4)
(1, 2, 3, 4)
```

`pulp.value(x)`

Returns the value of the variable/expression x, or x if it is a number



PuLP documentation by Pulp documentation team is licensed under a [Creative Commons Attribution-Share Alike 3.0 New Zealand License](#).